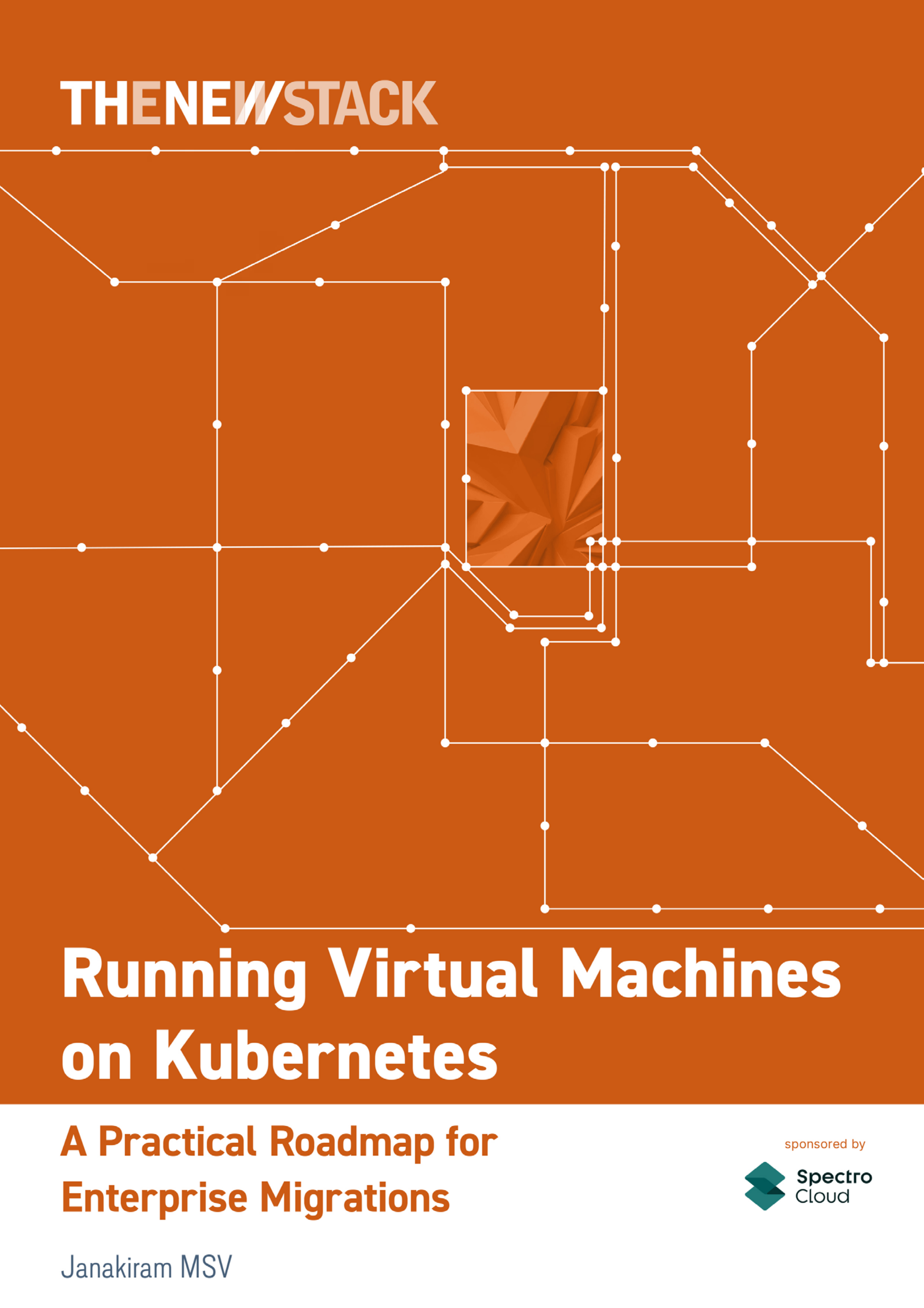


THE NEW STACK



Running Virtual Machines on Kubernetes

A Practical Roadmap for Enterprise Migrations

Janakiram MSV

sponsored by



The New Stack

Running Virtual Machines on Kubernetes: A Practical Roadmap for Enterprise Migrations

Alex Williams, Founder and Publisher

Ebook Team:

Ben Kubany, Project Manager

Diana Gonçalves, Graphic Designer

Janakiram MSV, Author

Michael Baker, Copy Editor

Susan Hall, Copy Editor

Vicki Walker, Ebook Editor

Supporting Team:

Andrew Tillery, Senior Marketing Manager

Benjamin Ball, Account Director

Heather Joslyn, Editor-in-Chief

Laila Bailey, Email Marketing Manager

Michelle Maher, Copy Editor

Shelley Chen, Social Media Manager

Tony Sala, Director of Sales

© 2025 The New Stack. All rights reserved.

20251118

Table of Contents

Sponsor 8

Introduction 9

 The Virtual Machine Challenge 10

 The KubeVirt Solution 10

 What You'll Learn 11

 Technical Foundation 11

 People and Process Transformation 11

 Migration Strategies 11

 Operational Excellence 11

 A Path Forward 12

Running VMs in Your Kubernetes Environment 13

 The VMware Landscape 14

 The Current Reality 15

 Why Kubernetes for VMs Makes Sense 17

 Unified Operations 17

 Consistent Infrastructure 17

 Open Standards 17

 Resource Efficiency 17

 KubeVirt as the Bridge to Cloud Native 18

 Alternative Approaches 19

 Hyperconverged Infrastructure (HCI) 19

 Cloud Infrastructure as a Service (IaaS) 19

 Complete Application Refactoring 20

- Practical Migration Strategies20
 - Gradual Transition20
 - Hybrid Operations20
 - Skills Reuse 21
- Managing the Transition 21
 - Skills Development..... 21
 - Operational Changes 21
 - Change Management 21
- Looking Forward22
- KubeVirt Fundamentals: Bridging VMs and Containers..... 23
 - Architecture Overview: How KubeVirt Extends Kubernetes24
 - Key Components and Their Roles25
 - VM Life-Cycle Management in Kubernetes29
 - Creating VMs29
 - Power Management29
 - Live Migration29
 - Template and Image Management 30
 - Networking, Storage, and Security Considerations 30
 - Storage Architecture..... 30
 - Network Configuration 31
 - Security Framework 31
 - Integration Considerations32

- Why KubeVirt Alone Isn't Enough 33
- Commercial Solutions and the Build vs. Buy Decision 36
 - The "Build" Approach (DIY) 36
 - The "Buy" Approach (Commercial Platforms) 36
 - Spectro Cloud Palette With Virtual Machine Orchestrator (VMO) 37
 - Red Hat OpenShift Virtualization 39
 - Platform9 39
 - SUSE Virtualization (Formerly SUSE Harvester) 40
- Deployment Infrastructure Choices 41
- Real-World Deployment Patterns 43
- Migration Planning and Assessment 45**
 - Understanding the Operational Shift 46
 - Application Deployment Patterns 47
 - Building Transition Readiness 48
 - Operational Practice Evolution 49
 - Skills Assessment and Team Structure 50
 - Skill Gap Analysis 51
 - Declarative Configuration 51
 - Container and Pod Concepts 51
 - Kubernetes Tooling 51
 - API-First Workflows 51
 - Organizational Considerations 52
 - Building Migration Readiness 52

- Change Management by Role54
- Mapping the VMware Stack to Cloud Native Equivalents56
- Tools and Techniques for Running VMs on Kubernetes 61**
 - Assessing Your VM Workloads62
 - Migration Approaches for VM Workloads63
 - Rehosting: The Fastest Path63
 - Replatforming: Gradual Modernization.....63
 - Kubernetes Native Tools: KubeVirt and CDI 64
 - KubeVirt 64
 - Containerized Data Importer (CDI) 64
 - Network and Storage Considerations.....65
 - Migration Automation Tools: Forklift and VM Migration Assistant65
 - Forklift: Open Source Migration.....65
 - Spectro Cloud VM Migration Assistant66
 - Selecting the Right Tool.....67
 - A Practical Migration Process67
 - Key Takeaways68
- Day 2 Operations and Optimization 69**
 - Unified Management for Mixed Workloads.....70
 - Applying Kubernetes Practices to VM Workloads 71
 - Monitoring and Optimization Strategies72
 - Working With VMs Within the Kubernetes Ecosystem.....73

Operational Considerations for VM Workloads 74

Key Takeaways 75

Conclusion and Key Takeaways 76

The Reality of Mixed Workloads 77

Practical Implementation 77

Key Technical Considerations 78

Operational Best Practices 79

Frequently Asked Questions 79

Moving Forward 80

For Further Reading 81

About the Author 82

Disclosure 83

Sponsor

We are grateful for the support of our sponsor for this ebook:



Spectro Cloud uniquely enables organizations to manage Kubernetes in production, at scale. Our Palette management platform gives effortless control of the full Kubernetes life cycle, across clouds, data centers, bare metal, and edge environments.



CHAPTER 1

Introduction

The enterprise IT landscape has been undergoing a fundamental transformation for years. Organizations across industries are adopting cloud native technologies, embracing containers, and modernizing their application delivery practices. This shift represents more than just technological change. It reflects a new approach to building and operating software that prioritizes agility, scalability, and resilience.

Kubernetes has emerged as the foundation of this transformation. What began as Google's internal container orchestration system has become the universal control plane for modern applications. Organizations are standardizing on Kubernetes — not just for new cloud native applications, but as their primary platform for all workloads, including legacy applications running in virtual machines (VMs).

Recent market events have accelerated this evolution. Broadcom's acquisition of VMware in 2023 created a pivotal moment, transforming a gradual modernization journey into an urgent business priority. The shift to subscription licensing, product bundling, and significant price increases forced many organizations to accelerate their cloud native adoption timelines.

However, this urgency has also created an opportunity. Organizations already invested in Kubernetes now have compelling business justification to consolidate their infrastructure around a single, open platform.

The Virtual Machine Challenge

Despite the momentum toward containerization, most enterprises still operate thousands of virtual machines. These workloads represent decades of application development and often include business-critical systems that cannot be easily rewritten or replaced. Legacy applications, commercial software with specific operating system (OS) requirements, and complex monolithic systems all require VM-based deployment models.

This creates a fundamental challenge for Kubernetes adopters. How do you standardize on cloud native infrastructure while maintaining the virtual machines that your business depends on?

Traditional approaches force organizations to maintain separate infrastructure stacks: VM workloads run on hypervisor platforms while containerized applications run on Kubernetes clusters. This division creates operational complexity, duplicated tooling, and inefficient resource utilization. Since each stack requires a unique skill set, different teams need to manage and operate the environments.

The KubeVirt Solution

KubeVirt addresses this challenge by extending Kubernetes to orchestrate virtual machines alongside containers. As a Cloud Native Computing Foundation (CNCF) project, KubeVirt provides a standardized way to run VMs using the same APIs, tools, and operational practices used for containerized workloads.

This approach enables a true infrastructure consolidation. Teams can manage both VMs and containers through a single control plane, using consistent security policies, networking configurations, and resource management practices. The result is simplified operations and more efficient resource utilization.

KubeVirt also provides a practical migration path. Organizations can move existing VMs onto their Kubernetes platform without requiring application changes. This “lift and shift” approach enables immediate infrastructure consolidation, allowing time to plan modernizing applications as separate initiatives.

What You'll Learn

This book provides comprehensive guidance for implementing VM management on Kubernetes using KubeVirt. It covers the technology, processes, and organizational considerations needed for successful adoption.

Technical Foundation

The book explores KubeVirt's architecture and core components, explaining how virtual machine concepts are translated into Kubernetes resources. You'll learn about VM life-cycle management, networking, storage, and security within the Kubernetes ecosystem.

People and Process Transformation

Successful implementation requires more than technology deployment. The book addresses the cultural shift from traditional VM operations to cloud native practices. This includes evolving

from graphical user interface (GUI)-based management to declarative, code-driven approaches using Infrastructure as Code (IaC) and GitOps methodologies.

The transformation follows the People, Process, and Technology framework. It requires upskilling teams, establishing new operational practices, and implementing the right technical solutions.

Migration Strategies

The book covers practical migration approaches, from simple VM rehosting to more advanced replatforming strategies. It examines both native Kubernetes tools and specialized migration platforms that automate the process at scale.

Operational Excellence

Beyond initial migration, the book explores ongoing operations in a unified environment. This includes monitoring,

optimization, security, and leveraging the broader Kubernetes ecosystem for advanced capabilities.

“This book serves as your guide through the entire process, from initial planning through long-term operations. It provides the practical knowledge needed to successfully implement VM management on Kubernetes and realize the full benefits of infrastructure consolidation.

A Path Forward

Organizations already committed to Kubernetes are well positioned to extend their platform to handle VM workloads. This approach builds on existing investments while simplifying infrastructure management.

The journey requires careful planning and execution, yielding substantial benefits. Unified infrastructure reduces operational complexity, improves resource efficiency, and provides a foundation for continued modernization.

This book serves as your guide through the entire process, from initial planning through long-term operations. It provides the practical knowledge needed to successfully implement VM management on Kubernetes and realize the full benefits of infrastructure consolidation.

The cloud native future isn't just about containers: It's about creating a unified, efficient, and agile platform that can handle all your workloads. KubeVirt makes that future possible today.



CHAPTER 2

Running VMs in Your Kubernetes Environment

Broadcom's acquisition of VMware in late 2023 marked a major milestone, not just as a change in ownership but as a significant event that has truly transformed the enterprise IT landscape. The recent changes in licensing, pricing, and product strategy have served as a strong motivator, encouraging a wide range of customers, including both small businesses and enterprises, to reevaluate their longstanding investments in VMware and explore alternative options.

For many enterprises, a once-comfortable, long-term partnership has transformed into a pressing and necessary reevaluation of their entire infrastructure strategy.

The VMware Landscape

Broadcom's acquisition of VMware has introduced significant changes for many organizations. The shift from perpetual licenses to mandatory subscriptions, product portfolio consolidation, and increased minimum core requirements have created cost pressures for many customers, particularly smaller organizations and those with distributed deployments.

While VMware remains a capable platform with millions of users, these licensing changes have accelerated interest in alternative approaches. For organizations already invested in Kubernetes, this presents an opportunity to consolidate infrastructure around an open platform that avoids vendor lock-in.

“The true power of KubeVirt lies in its role as a strategic enabler for a phased and derisked migration.

The Current Reality

According to “[The Voice of Kubernetes Experts Report 2024](#)” from Portworx, 58% of the respondents planned to migrate applications from VM management to Kubernetes.

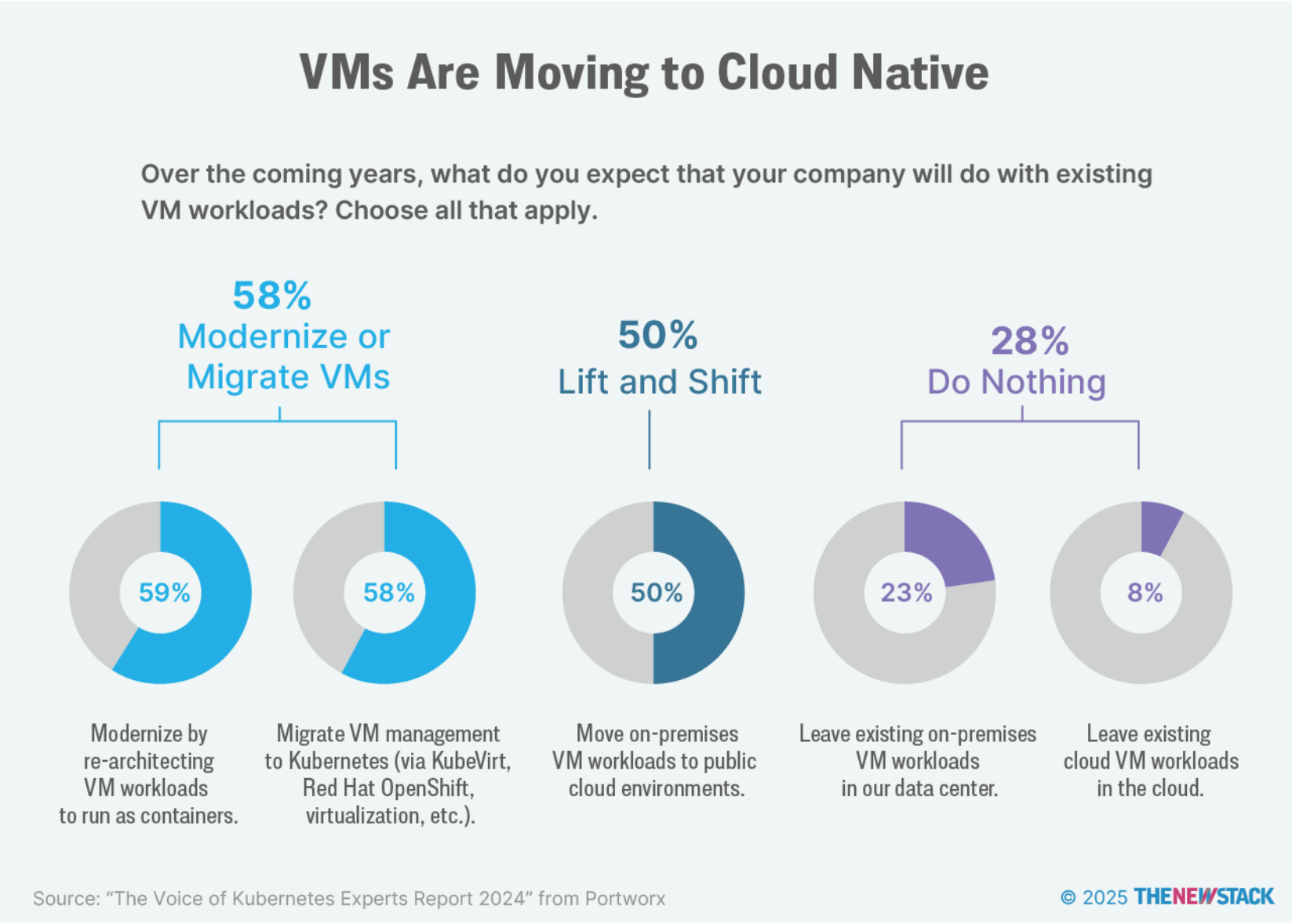


FIG 2.1 Nearly 60% of organizations say they plan to migrate applications from VM management to Kubernetes

More recently, Spectro Cloud’s “[2025 State of Production Kubernetes](#)” report indicated that it is becoming mainstream to use KubeVirt to support VM workloads. Almost a third (31%) of respondents plan to shift VMs to Kubernetes, while 52% expect their orchestration of VMs on Kubernetes to grow significantly over the next 12 months.

The true power of KubeVirt lies in its role as a strategic enabler for a phased and derisked migration. It offers a pragmatic middle ground between three extreme and often undesirable migration strategies.



FIG 2.2 Responses to the Spectro Cloud survey question: “Which of the following best describes your business’ strategy toward VM workloads?”

Why Kubernetes for VMs Makes Sense

For organizations already invested in Kubernetes, running VMs on the same platform offers several compelling advantages:

investing in industry standards rather than proprietary technologies, providing long-term flexibility and avoiding vendor lock-in.

Unified Operations

Managing containers and VMs through a single orchestration platform simplifies operations. Your teams can use the same tools, APIs, and workflows for all workloads, reducing complexity and training overhead.

Resource Efficiency

Kubernetes excels at resource optimization through intelligent scheduling and bin packing. Whether it is running containers or VMs, the platform maximizes hardware utilization by dynamically allocating resources based on actual demand.

Consistent Infrastructure

A single Kubernetes cluster can host both containerized microservices and traditional VM-based applications. This eliminates the need to maintain separate infrastructure stacks, reducing operational costs and complexity.

“Kubernetes excels at resource optimization through intelligent scheduling and bin packing.”

Open Standards

Kubernetes is an open source project under the CNCF. This ensures you're

KubeVirt as the Bridge to Cloud Native

KubeVirt is a CNCF project in the incubation stage that extends Kubernetes to orchestrate virtual machines alongside containers. It enables you to manage VMs using the same Kubernetes APIs, tools, and operational practices you're probably already familiar with.

This approach addresses a common challenge: what to do with legacy applications that cannot be easily containerized. These include applications with specific OS dependencies, commercial software with restrictive licensing, or systems that require significant refactoring for containerization.

KubeVirt enables a “lift and shift” approach. If your existing VMs are in a supported format, you can move them directly onto your Kubernetes platform without modifying the applications; otherwise you can use tools such as Containerized Data Importer (CDI) and Forklift to convert them into a supported format. (See [Chapter 5](#) for information about these tools.) This

approach provides immediate infrastructure consolidation while allowing time to plan application modernizations as separate projects.

“This approach addresses a common challenge: what to do with legacy applications that cannot be easily containerized.”

Alternative Approaches

Several other options exist for organizations looking to modernize their VM infrastructure:

Hyperconverged Infrastructure (HCI)

Platforms like Nutanix provide integrated compute, storage, and networking in single appliances. These solutions simplify VM operations and offer strong performance guarantees.

However, HCI platforms have notable drawbacks. They typically use proprietary management tools and APIs, creating vendor lock-in similar to traditional virtualization platforms. The integrated hardware model can lead to higher costs and reduced flexibility in choosing compute, storage, or networking components independently. Organizations also face challenges when integrating HCI environments with existing Kubernetes infrastructure, often requiring separate operational teams and tool sets.

Cloud Infrastructure as a Service (IaaS)

Moving VMs to public cloud IaaS services like Amazon Web Services (AWS) EC2 or Azure Virtual Machines can solve immediate hardware concerns and provide access to extensive cloud services.

The disadvantages include potentially higher long-term costs, especially for steady-state workloads that run continuously. Organizations often experience “cloud sprawl” where costs become difficult to predict and control. This approach also doesn’t modernize the operational model, meaning teams continue using the same VM-centric practices rather than adopting cloud native approaches. Data egress charges and network latency can become significant issues for applications with high data transfer requirements or strict performance needs.

Practical Migration Strategies

Complete Application Refactoring

Rewriting monolithic applications as containerized microservices represents the ideal cloud native end state and unlocks the full benefits of modern platform capabilities.

However, this approach requires substantial upfront investment in time, engineering resources, and organizational change. The risk of introducing bugs or performance regressions during refactoring can be significant for business-critical applications. Many organizations lack the deep institutional knowledge needed to safely refactor legacy systems, especially those built years ago by teams that are no longer with the company. The business often cannot justify the cost and timeline for refactoring when existing applications work reliably, making this approach impractical for immediate infrastructure consolidation needs.

There are several best practices that can simplify and streamline migrating VMs to Kubernetes.

Gradual Transition

Start by identifying VM workloads that are good candidates for migration. These typically include development environments, test systems, and applications with flexible uptime requirements. Success with these initial workloads builds confidence and expertise for larger migrations.

Hybrid Operations

KubeVirt allows you to run VMs and containers on the same infrastructure. This means you can maintain critical VM-based applications while gradually containerizing others. Both workload types benefit from the scheduling, networking, and storage capabilities offered by Kubernetes.

Managing the Transition

Skills Reuse

Your existing Kubernetes knowledge directly applies to VM management through KubeVirt. The same `kubectl` commands, YAML manifests, and operational procedures work for both containers and VMs.

Success with a VM-to-Kubernetes migration depends on more than technology choices. The most critical factors are organizational and cultural.

Skills Development

While Kubernetes knowledge transfers well to KubeVirt, teams need time to learn about VM-specific operations within the platform. Invest in training and hands-on practice with noncritical workloads first.

Operational Changes

Moving from GUI-based VM management to API-driven Kubernetes operations represents a significant shift. Teams accustomed to VMware vSphere will need to adapt to `kubectl` and YAML-based workflows.

Change Management

Like any infrastructure transformation, this migration process requires strong

“KubeVirt allows you to run VMs and containers on the same infrastructure. This means you can maintain critical VM-based applications while gradually containerizing others.”

leadership support and clear communication about benefits. Start with pilot projects that demonstrate value and build momentum for larger initiatives.

“The question isn't whether to modernize your infrastructure, but rather how to do it in a way that maximizes business value while minimizing risk.

Looking Forward

The trend toward Kubernetes as a unified platform for all workloads is accelerating. Organizations that embrace this option early gain operational advantages and avoid the complexity of managing multiple infrastructure stacks.

KubeVirt offers a practical approach to embracing this trend without requiring immediate application rewrites. By running VMs on Kubernetes, you can consolidate infrastructure, improve resource utilization, and maintain operational consistency across all workloads.

The question isn't whether to modernize your infrastructure, but rather how to do it in a way that maximizes business value while minimizing risk. For organizations already committed to Kubernetes, extending the platform to handle VMs through KubeVirt offers the most pragmatic approach to infrastructure consolidation.

CHAPTER 3

KubeVirt Fundamentals: Bridging VMs and Containers

As organizations chart their course away from traditional virtualization, KubeVirt emerges not just as a tool but as a foundational technology that makes a phased, pragmatic migration possible. It acts as a bridge, enabling the coexistence of legacy virtual machines and modern containers on a single, unified platform: Kubernetes.

Understanding KubeVirt's architecture and capabilities is the first step in leveraging it to derisk the migration process, consolidate infrastructure, and accelerate the journey to a cloud native operating model.

This chapter explores the technical foundations, practical limitations, and real-world implementation patterns that are essential for infrastructure evaluation.

Architecture Overview: How KubeVirt Extends Kubernetes

KubeVirt’s design philosophy is straightforward and builds firmly on aspects where Kubernetes already excels. Instead of creating a new, parallel orchestration system for virtual machines, KubeVirt extends the reputable Kubernetes API and control plane, enabling it to manage VMs as native resources. It efficiently delegates core functions like scheduling, networking, and storage directly to Kubernetes, while layering on the specific logic required for virtualization.

At its heart, a KubeVirt VM is simply a process running inside a standard Kubernetes pod. This approach allows VMs and containers to run side by side on the same worker nodes, communicate over the same network, and use the same storage resources, all managed through a single pane of glass.

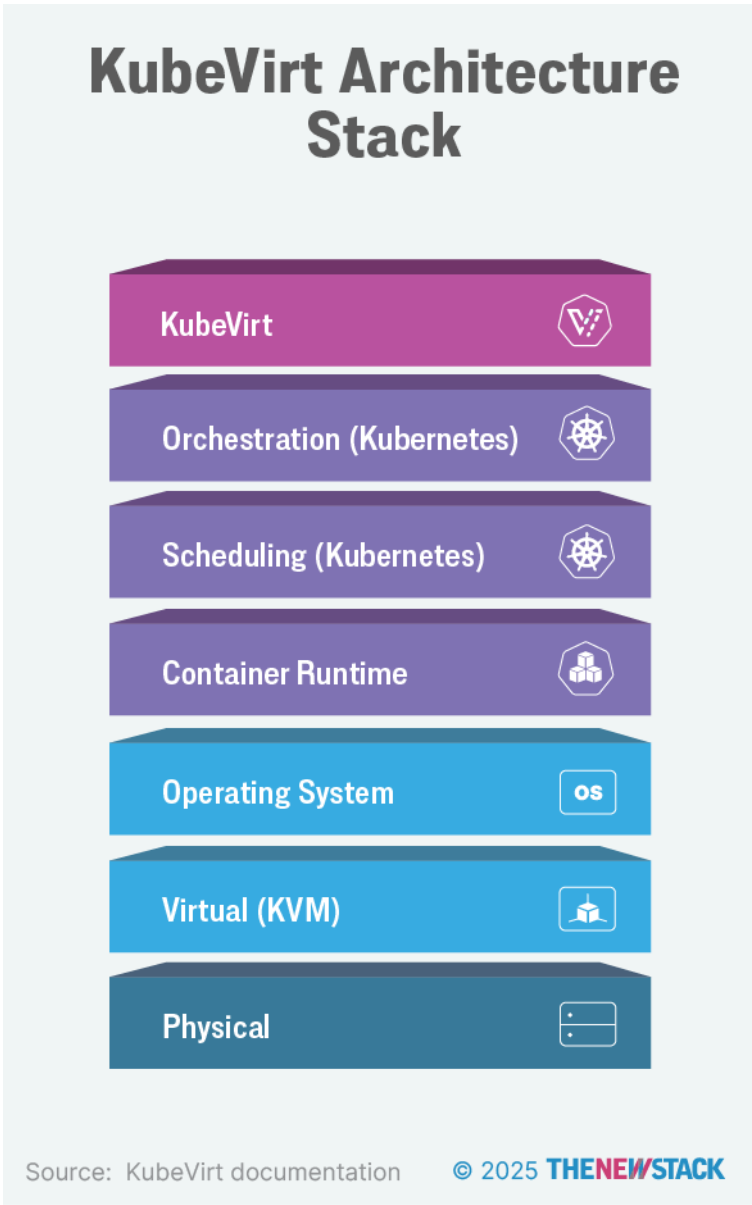


FIG 3.1 KubeVirt adds virtualization capabilities to Kubernetes.

To achieve this, KubeVirt introduces three main types of components into the cluster:

1. **Custom Resource Definitions (CRDs):** These are extensions to the Kubernetes API that define new object types. KubeVirt adds several CRDs, most notably `VirtualMachine` and `VirtualMachineInstance` (VMI). This allows administrators to define a `VirtualMachine` using a declarative YAML manifest, just as they would for any other Kubernetes object, such as a pod.
2. **Controllers:** These are cluster-wide components that contain the business logic for managing the new CRDs. They run as pods and watch the Kubernetes API for changes.
3. **Daemons:** These are node-specific agents, deployed as a `DaemonSet`, that are responsible for managing the VM life cycle on each worker node in the cluster.

Key Components and Their Roles

The interplay between KubeVirt's components creates a seamless virtualization layer within Kubernetes. While an operator can install all necessary components, understanding the individual roles of these components is key to troubleshooting and effective management.

- **VirtualMachine and VMI:** These are the two primary CRDs that users interact with. The `VirtualMachine` object represents the persistent, desired state of a virtual machine. It can be started and stopped while retaining its configuration and data. The `VirtualMachineInstance` represents the actual running instance of that `VirtualMachine`. A VMI is more ephemeral, existing only while the `VirtualMachine` object is in a running state, and is tightly coupled to the pod that hosts it.

- **virt-api server:** This serves as the HTTP API entry point for all virtualization flows, acting as an interface for the operations of VMI CRDs. It validates, processes, and persists VMI and VirtualMachine resource definitions into Kubernetes, allowing the rest of the KubeVirt control plane to react.
- **virt-controller:** This is the central, clusterwide controller. Its primary job is to watch for the creation of new VMI objects. When a VMI is defined, virt-controller creates a corresponding pod that will ultimately host the VirtualMachine process. It handles high-level operations and orchestrates complex actions, such as live migrations.
- **virt-handler:** This is a DaemonSet, meaning an instance that runs on every worker node. It acts as the node-specific agent. When a VM's pod is scheduled onto its node, virt-handler takes over. It communicates with the virt-launcher inside the pod to perform all the necessary operations to start, stop, and manage the VM process on that specific host.
- **virt-launcher:** For every running VM, there is a dedicated pod, and the primary container within that pod runs the virt-launcher component. This component is the final link in the chain. It receives instructions from virt-handler and uses a local [libvirt](#) instance to start and manage the actual [QEMU](#)/Kernel-based Virtual Machine ([KVM](#)) process that constitutes the virtual machine. It also ensures a graceful shutdown by trapping signals from Kubernetes and passing them to the VM process.
- **libvirt:** This is a hypervisor management daemon running inside the virt-launcher

container. It exposes a control interface to QEMU/KVM, handling VM life-cycle commands such as start, stop, pause, resume, and migrate. It abstracts away the complexities of interacting directly with QEMU by offering a stable API.

- **QEMU:** This is a user-space emulator and virtualizer invoked by libvirt inside the virt-launcher container. QEMU emulates the VM’s hardware environment and executes the guest operating system with hardware acceleration through KVM when available. It handles device emulation, I/O operations, and CPU virtualization.

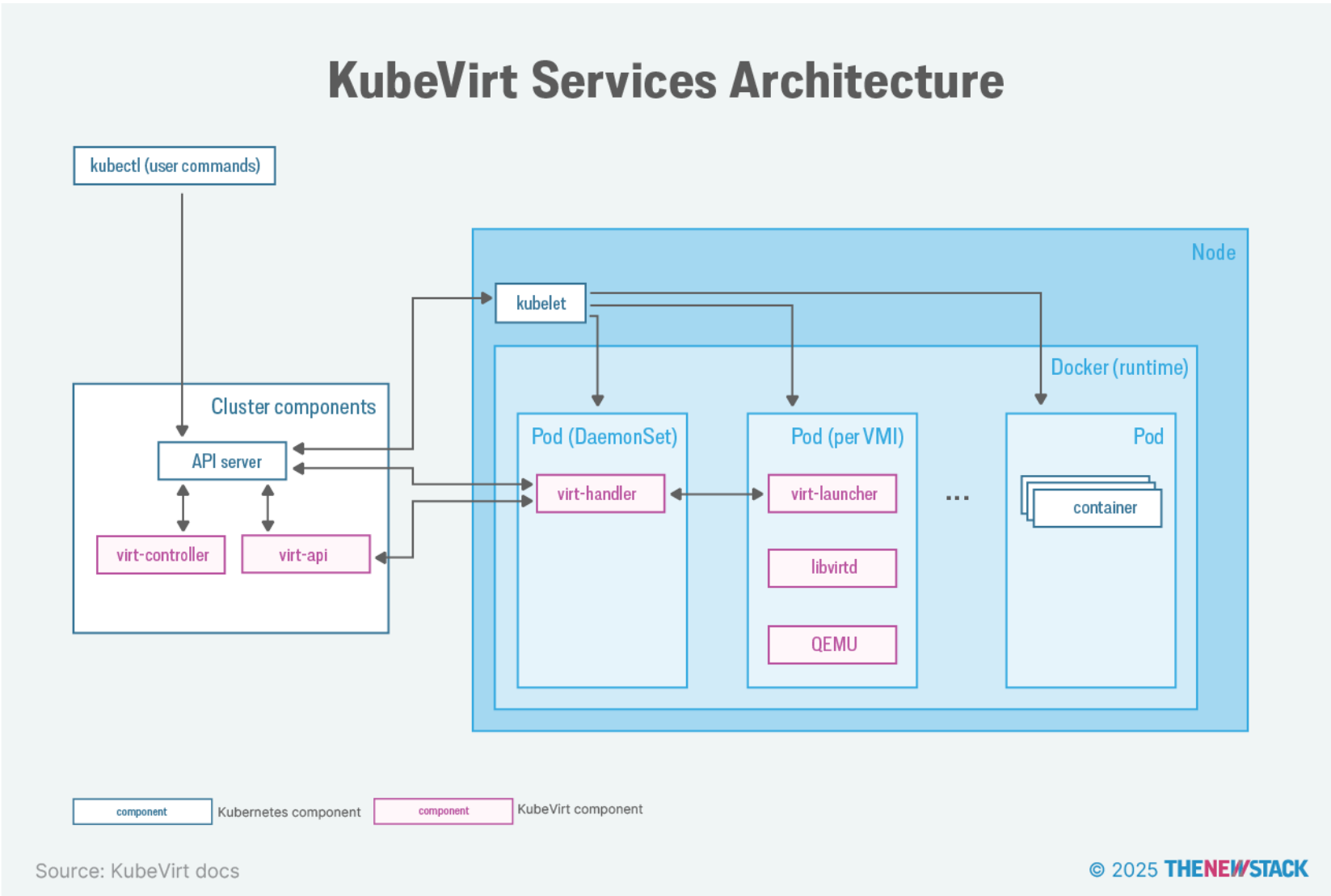


FIG 3.2 Communication and storage of additional controllers and daemons.

Key KubeVirt Components	
Component	Responsibility
virt-api	Exposes the virtualization API to users.
virt-controller	Handles life-cycle logic, including powering on/off, resizing, and migration orchestration.
virt-handler	Node-level reconciliation: creates domains and manages devices via libvirt/QEMU.
virt-launcher	An isolated pod that runs QEMU/KVM for each VM instance.
libvirt	Hypervisor management daemon that provides a control interface to QEMU/KVM for VM life-cycle and device operations.
QEMU	User-space emulator and virtualizer that runs the VM process, emulating hardware devices and executing guest instructions with KVM acceleration when available.
VirtualMachine and VMI	CRDs that represent the image and running instance of a VM.

This architecture allows KubeVirt to translate familiar virtualization concepts into the Kubernetes world, treating a

VM as just another workload to be managed by the cluster’s powerful orchestration engine.

VM Life-Cycle Management in Kubernetes

Managing virtual machines in KubeVirt uses the same declarative, code-driven approach as other Kubernetes resources. By default, VM operations are performed through YAML manifests and command-line interface (CLI) tools rather than graphical interfaces, though GUI options are also available for those who prefer them.

KubeVirt provides two primary tools for VM management. The standard `kubectl` command works with VM resources just like any other Kubernetes object. For VM-specific operations that Kubernetes does not natively support, `virtctl` extends the tool kit with specialized commands for VM console access, live migration, and power management.

Creating VMs

Virtual machines are defined using YAML manifests with the `VirtualMachine` custom resource. These files declaratively specify the VM's desired configuration, including CPU allocation, memory size, disk attachments, and

network interfaces. Once applied to the cluster, Kubernetes ensures the VM matches this specification.

Power Management

VM power states can be controlled through multiple approaches. The `virtctl` command provides immediate control, with commands like `virtctl start` and `virtctl stop` for specific virtual machines. For declarative management, you can modify the `runStrategy` field in the VM manifest to values like `Running` or `Halted`, allowing Kubernetes to maintain the desired power state automatically.

Live Migration

KubeVirt supports live migration of running virtual machines between cluster nodes with zero downtime. You can initiate migration using the `virtctl migrate` command or by creating a `VirtualMachineInstanceMigration` resource through `kubectl`. The primary

Networking, Storage, and Security Considerations

requirement is that VM disks must be stored on shared storage that is accessible from multiple nodes, typically using Persistent Volume Claims (PVCs) with ReadWriteMany (RWX) access mode.

Template and Image Management

The CDI handles VM disk images and templates within KubeVirt. CDI can import existing disk images into PVCs, making them available for VM creation. These imported images can serve as base templates that are cloned to create new VMs, providing a foundation for standardized VM deployment workflows.

Building a production-ready KubeVirt platform requires careful planning around networking, storage, and security. Each area builds upon Kubernetes foundations while adding VM-specific capabilities and requirements.

Storage Architecture

KubeVirt uses Kubernetes native storage concepts for VM disk management. Virtual machines use PVCs to request storage rather than using traditional data stores. The storage characteristics, such as performance profiles and access modes, are defined through StorageClass objects, which connect to underlying storage systems via Container Storage Interface (CSI) drivers.

Live migration requires storage that multiple nodes can access simultaneously. This typically involves using StorageClass objects that provide RWX volumes through technologies such as Network File System (NFS),

CephFS, or distributed storage systems. For high-performance workloads such as databases, PVCs can be configured with `volumeMode` set to `Block`, providing raw block devices directly to VMs for optimal input/output (I/O) performance.

KubeVirt also supports storage operations, such as cloning and snapshots, when the underlying CSI driver provides these capabilities. This enables workflows such as creating VM templates from existing disks or taking point-in-time backups of running systems.

Network Configuration

Virtual machines connect to the Kubernetes pod network by default using a masquerade binding, which provides Network Address Translation (NAT) access to the cluster network. This approach integrates VMs seamlessly with existing Kubernetes networking and service discovery mechanisms.

More complex networking scenarios require additional tools. Multus serves as a CNI meta-plugin that enables pods and their contained VMs to attach to multiple networks simultaneously. This capability supports use cases like connecting VMs to specific virtual LANs (VLANs) through bridge networks or providing high-performance connectivity via Single Root I/O Virtualization (SR-IOV) pass-through devices.

The choice of Container Network Interface (CNI) plugin has a significant impact on the available networking features. Different CNI implementations offer varying levels of functionality to meet advanced networking requirements, including network segmentation, traffic shaping, and performance optimization.

Security Framework

KubeVirt inherits Kubernetes security models while extending them for virtual machine workloads. Namespaces

provide the primary isolation boundary, grouping related VMs and containers while controlling their access to cluster resources. This approach creates logical separation, similar to organizing VMs into folders or resource pools.

Role-based access control (RBAC) defines granular permissions for VM management. RBAC policies specify which users or service accounts can create, delete, modify, or access VMs within specific namespaces. This enables fine-grained delegation of administrative responsibilities across different teams or projects.

Network policies control traffic flow between VMs and other cluster workloads. These policies provide basic network segmentation capabilities, though their effectiveness depends entirely on the CNI plugin implementation. Some CNI solutions offer more advanced policy enforcement and monitoring capabilities than others.

Pod Security Standards and admission controllers can enforce security policies

on VM workloads just like containerized applications. This includes restrictions on privileged operations, resource limits, and security contexts that govern how VMs operate within the cluster.

Integration Considerations

VM management through KubeVirt inherits many advantages from the Kubernetes platform. Resource management uses the same quota and limit systems as containers. Network policies function consistently across both VMs and pods. Storage management adheres to standard Kubernetes patterns, using persistent volumes and storage classes.

The declarative model means VM configurations can be version-controlled, reviewed, and deployed through standard DevOps practices. Teams can apply the same GitOps workflows used for containerized applications to their VM infrastructure, bringing consistency to operations across different workload types.

Why KubeVirt Alone Isn't Enough

The convergence of VM and container workloads on a single platform creates opportunities for unified management approaches. Storage policies can apply consistently across both workload types. Network segmentation strategies can encompass VMs and pods within the same policy framework. Security controls benefit from centralized management and consistent enforcement mechanisms.

However, this integration also requires careful planning to ensure that VM-specific requirements, such as live migration, console access, and compatibility with legacy applications, are adequately addressed within the broader Kubernetes operational model.

While KubeVirt provides the core engine for running VMs on Kubernetes, the open source project by itself is not a feature-complete, enterprise-grade replacement for a mature platform like VMware vSphere. Organizations adopting KubeVirt must understand that it is a foundational component that requires a [larger ecosystem of tools](#) to meet production requirements.

According to the [“2025 State of Production Kubernetes” report](#), 76% of users try KubeVirt, using the open source project to test its functionality, but only 32% deploy it to production. Production deployments are much more likely to rely on commercial KubeVirt-based solutions.

“Adopting KubeVirt is not a simple product swap but a strategic platform engineering effort.

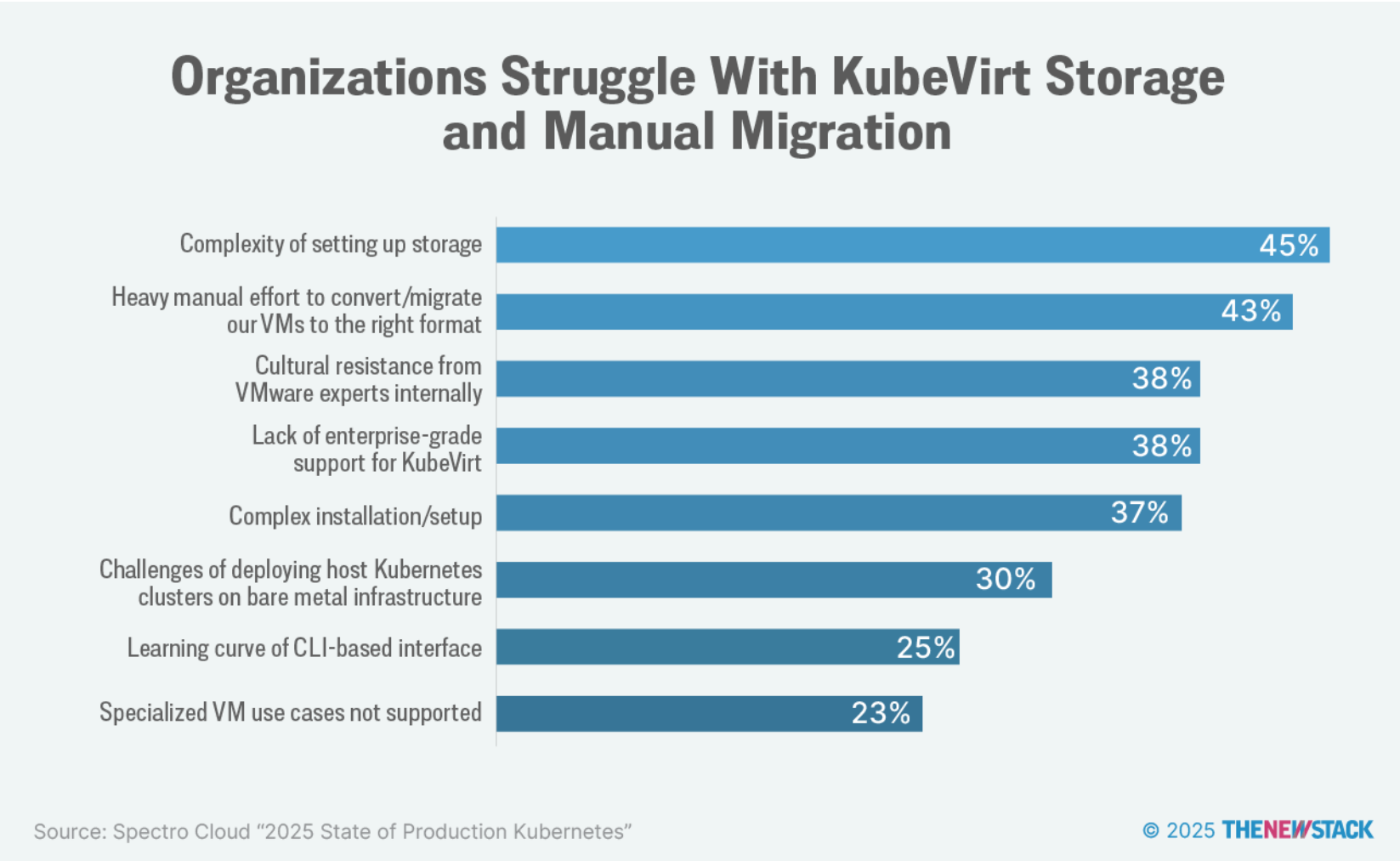


FIG 3.3 Responses to the Spectro Cloud survey question: “What challenges did you face when trying KubeVirt?”

Customers should be aware of the primary gaps in the upstream KubeVirt project:

- **Limited life-cycle management:** Out of the box, KubeVirt provides basic life-cycle operations: create, start, stop, and delete. It lacks the rich, built-in automation for tasks like advanced templating, cloning, and comprehensive disaster recovery that enterprises rely on. While some of these features are possible, they often depend on integrating other tools, such as

the CDI, and are less mature than their vSphere counterparts.

- **Storage and networking dependencies:** By design, KubeVirt does not have built-in storage or networking solutions. It instead relies entirely on the underlying Kubernetes CNI and CSI drivers. This means that critical features, such as storage snapshots, volume expansion, live migration, and network segmentation, are only as good as the third-party plugins you choose.

- **Operational tooling and user interface (UI):** While open source KubeVirt does not come with a GUI, integrated monitoring dashboards, or advanced observability tools, commercial solutions add user interface elements to perform typical operations. A production-ready stack requires integrating observability solutions such as Prometheus and Grafana, as well as developing operational workflows around command-line tools (kubectl, virtctl) and YAML manifests.
- **Enterprise-grade support and security:** As an open source project, KubeVirt relies on community support. For mission-critical workloads, this may not be sufficient. Furthermore, while it inherits Kubernetes' security primitives, KubeVirt lacks built-in features for compliance reporting and auditing, which are often required in regulated industries such as finance and government.

Adopting KubeVirt is not a simple product swap but a strategic platform engineering effort. It requires assembling and integrating multiple components to build a complete, production-ready virtualization stack.

“Organizations adopting KubeVirt must understand that it is a foundational component that requires a larger ecosystem of tools to meet production requirements.”

Commercial Solutions and the Build vs. Buy Decision

Organizations looking to adopt KubeVirt face a strategic choice: build a platform from open source components (DIY) or buy a commercially supported, preintegrated solution.

The ‘Build’ Approach (DIY)

This path involves starting with a base Kubernetes cluster and manually integrating KubeVirt along with all the necessary ecosystem components for storage (Ceph, Longhorn), networking (Multus, Calico), monitoring (Prometheus, Grafana), and data protection (CloudCasa).

Pros: Maximum flexibility and control, no vendor licensing fees for the core components, and avoiding vendor lock-in.

Cons: Requires deep in-house Kubernetes expertise, a significant investment in engineering time for integration and ongoing maintenance, and a longer time to deploy and

operationalize. The organization assumes full responsibility for security, patching, and support.

The ‘Buy’ Approach (Commercial Platforms)

This path involves adopting a curated platform that bundles KubeVirt into a complete, enterprise-ready solution with a unified management experience and professional support.

Pros: Faster time to value, reduced operational complexity, enterprise-grade support and service-level agreements (SLAs), and integrated features that are tested and validated to work together. This eliminates the need to hire or train a large team of specialized Kubernetes experts.

Cons: Incurs subscription costs, may offer less customization than a DIY approach, and introduces a degree of vendor dependency.

Spectro Cloud Palette With Virtual Machine Orchestrator (VMO)

[Palette](#) is a modern enterprise Kubernetes management platform that provides unified, full-stack life-cycle management for clusters spanning clouds, bare metal, and edge environments. Its Virtual Machine

Orchestrator (VMO) capability extends open source KubeVirt and other CNCF projects to let organizations run VMs and containers side by side as first-class citizens within the same Kubernetes clusters, fully managed through a unified interface, which includes a GUI, a CLI, an API, and IaC through a Terraform provider.

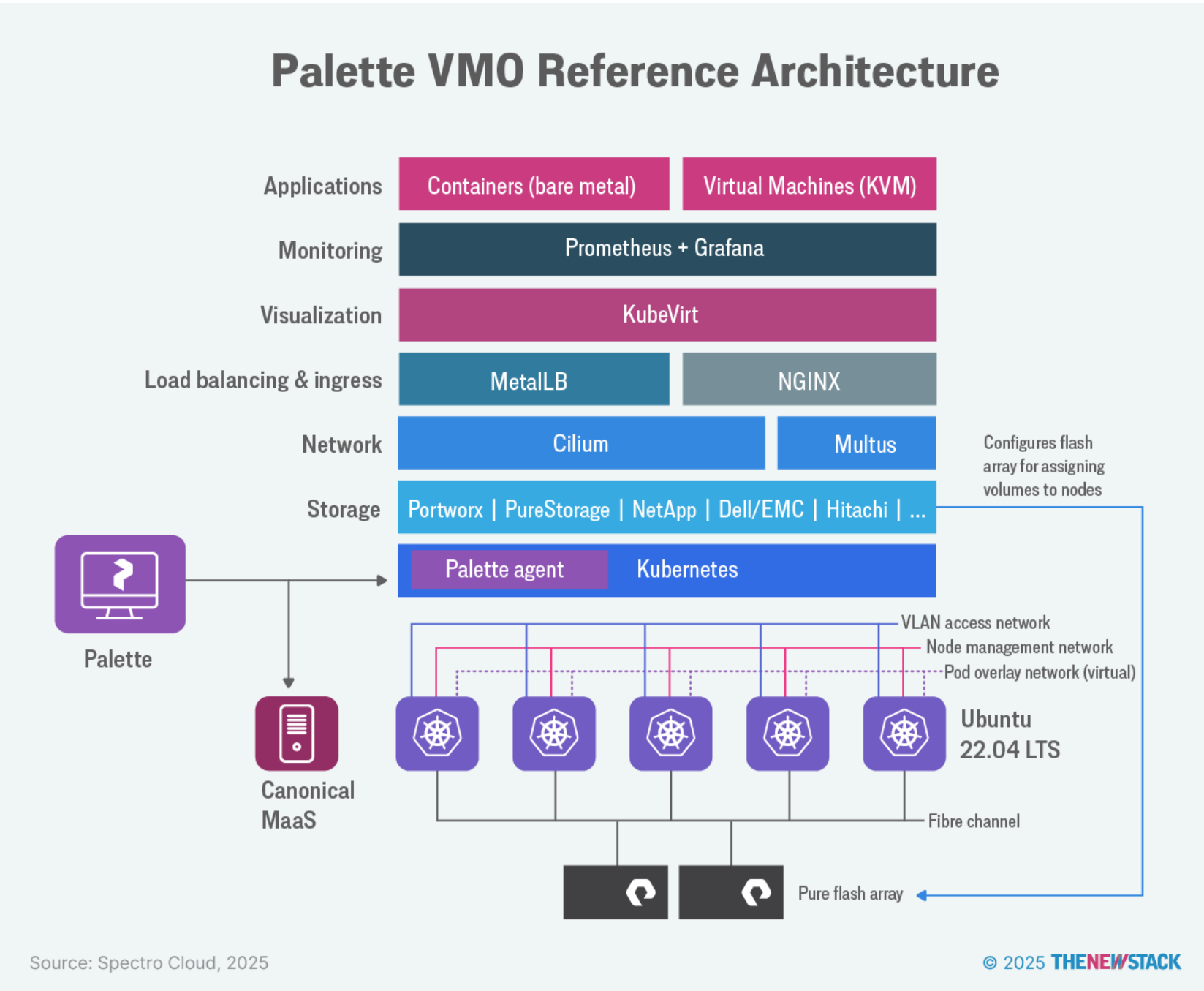


FIG 3.4 Core technologies in the VMO reference architecture.



FIG 3.5 Configuration of each layer in the cluster profile for the VMO reference architecture.

Unlike traditional hypervisors or HCI solutions, Palette is infrastructure-agnostic, supporting a wide range of storage, networking, and bare metal provisioning tools such as Longhorn, Portworx, Cilium, and Canonical Metal as a Service (MaaS).

With Palette, VMs benefit from declarative, GitOps-driven management, live migration, dynamic resource rebalancing, snapshots, and integration with enterprise features like monitoring and RBAC. VM Migration Assistant provides an on-ramp for migrating workloads from an existing VMware platform, accelerating time to value while reducing risk and operational overhead.

Palette is available as a Software as a Service (SaaS) offering or for self-hosted installations, including air-gapped environments. It stands out for its flexible, transparent pricing, with VMO included as a core feature for all customers. As a result, organizations modernizing legacy workloads or pursuing a hybrid infrastructure strategy benefit from cost efficiency, operational simplicity, and freedom from vendor lock-in, all while leveraging the

openness and scalability of Kubernetes and the CNCF ecosystem.

Red Hat OpenShift Virtualization

A feature of the OpenShift Container Platform, OpenShift Virtualization is one of the most mature commercial KubeVirt offerings. It provides an integrated solution with a unified UI, advanced features, and enterprise support from Red Hat. It also includes tools to simplify migration from other hypervisors.

Platform9

Platform9 Managed KubeVirt is a managed KubeVirt solution that launched in 2021. It offers unified VM and container management on Kubernetes. Platform9 provides two distinct approaches: its KubeVirt-based solution for cloud native environments and Private Cloud Director as a comprehensive VMware replacement using KVM/OpenStack. Both offerings include enterprise features like live migration, high availability, and VMware DRS-equivalent capabilities.

SUSE Virtualization (Formerly SUSE Harvester)

SUSE Virtualization, formerly branded as SUSE Harvester, is a mature open source hyperconverged infrastructure solution. It is built on KubeVirt, which provides it with access to Kubernetes (RKE2) and Longhorn distributed storage. Key features include built-in high availability, live migration, GPU

support, and comprehensive monitoring through Grafana/Prometheus. Its deep Rancher integration enables unified management of VMs and containers through a single Kubernetes control plane.

The table below compares and contrasts upstream KubeVirt with a commercial platform, such as those mentioned above.

KubeVirt: DIY vs. Commercial Platform		
Factor	Build (DIY Open Source)	Buy (Commercial Platform)
Cost	No direct license fees, but requires substantial upfront and ongoing investment in engineering talent, operational time, and specialized expertise.	Subscription fees, but potentially lower total cost of ownership (TCO) due to reduced internal effort.
Control and Customization	Full control over every component and configuration.	Less granular control; customization is within the vendor's supported framework.
Time to Deploy and Operationalize	Slower; requires significant time for design, integration, and validation.	Faster; provides a preintegrated, off-the-shelf, production-ready platform out of the box.
Required Expertise	High; requires a dedicated team with deep expertise in Kubernetes, storage, and networking.	Lower; the vendor handles the platform's complexity, allowing teams to focus on workloads.
Support and Maintenance	Community-based support; the internal team is responsible for all patching and upgrades.	Enterprise-grade support with SLAs; vendor manages platform maintenance and security.

Deployment Infrastructure Choices

If you have a platform engineering team with deep Kubernetes skills, a DIY KubeVirt approach may suffice. If you need audited support, validated stack components (storage/networking), certified performance, or streamlined migration, commercial or curated solutions can save months of engineering effort.

“If you have a platform engineering team with deep Kubernetes skills, a DIY KubeVirt approach may suffice.”

Organizations deploying KubeVirt must select the appropriate underlying infrastructure based on their operational requirements, existing investments, and technical constraints. Each deployment model presents unique advantages and considerations that influence the overall platform architecture.

While bare metal servers may provide the highest performance and most direct hardware access for KubeVirt workloads, the real advantage lies in avoiding the license costs of a commercial hypervisor. This approach eliminates virtualization overhead and enables features like SR-IOV and GPU pass-through for demanding applications. If you opt for bare metal, you also need to plan for how to bootstrap and manage the bare metal servers themselves — something few Kubernetes management platforms have an elegant solution for. Spectro Cloud’s Palette simplifies bare metal deployments through integration with provisioning tools like Canonical MaaS and supports automated OS installation and cluster bootstrapping.

Cloud IaaS platforms provide elastic scalability and reduce the operational burden for KubeVirt deployments. Organizations can leverage cloud provider networking and storage services while maintaining consistency with on-premises deployments. Palette provides unified management across major cloud providers, abstracting infrastructure differences through consistent cluster profiles.

Edge devices with limited CPU, memory, and storage require carefully optimized KubeVirt configurations. These deployments often prioritize specific workloads and minimize resource overhead. Palette addresses edge requirements through lightweight Kubernetes distributions and automated workload placement based on resource availability.

Air-gapped environments in regulated industries or secure facilities require completely disconnected operations. These deployments need local image registries, offline updates, and self-contained management tools. Palette

supports air-gapped installations with local artifact repositories and disconnected cluster management capabilities.

“Edge devices with limited CPU, memory, and storage require carefully optimized KubeVirt configurations.”

Real-World Deployment Patterns

Organizations are adopting KubeVirt to solve specific, practical challenges, leading to several common deployment patterns:

- **Phased modernization platform:** The most common pattern involves using KubeVirt to “lift and shift” legacy applications that are difficult to containerize directly onto the Kubernetes platform. This enables infrastructure consolidation and a unified control plane, allowing the complex work of refactoring the application into microservices to be done iteratively over time.
- **Hybrid applications:** This pattern involves deploying applications where some components are modern, containerized microservices, while others, such as a stateful database or a licensed third-party application, run in a KubeVirt VM. Both parts coexist in the same cluster, often in the same namespace, and communicate seamlessly over the same network.
- **CI/CD and testing environments:** Teams use KubeVirt to dynamically provision and destroy ephemeral, full-OS virtual machines as part of their automated CI/CD pipelines. This is useful for tasks like building custom OS images, running end-to-end tests that require specific kernel modules, or validating software in a realistic environment.
- **Edge and telecommunications:** In resource-constrained edge locations, KubeVirt enables the co-location of traditional virtual network functions (VNFs), which run in VMs, and modern, cloud native, and containerized network functions (CNFs) on the same hardware. This reduces the physical footprint and simplifies management through a centralized Kubernetes orchestrator.

- High-security multitenancy: For service providers or large enterprises, KubeVirt can provide strong, kernel-level isolation between tenants by running each tenant's workloads inside a dedicated VM. This pattern is also used to run entire nested Kubernetes clusters for development or sandboxing purposes — a model sometimes referred to as “Kubernetes-in-Kubernetes.”

By understanding these fundamentals, organizations can make informed decisions about how to best leverage KubeVirt as a strategic asset in their migration away from legacy virtualization and toward a more agile, resilient, and cost-effective cloud native future.

“ Palette provides unified management across major cloud providers, abstracting infrastructure differences through consistent cluster profiles.

CHAPTER 4

Migration Planning and Assessment

Moving VM workloads from VMware vSphere to Kubernetes using KubeVirt involves changes in how teams deploy and manage applications. While the underlying infrastructure technology shifts, the more significant impact often lies in adapting operational practices and team workflows to work effectively with cloud native tooling.

This chapter explores the practical aspects of migration planning, including assessing organizational readiness, building the right team capabilities, and managing the transition from VM-centric operations to Kubernetes native approaches.

Understanding the Operational Shift

Organizations already running Kubernetes for containerized applications have established cloud native practices. However, extending these practices to VM workloads often reveals gaps in the way traditional and cloud native operations intersect.

According to Spectro Cloud’s “2024 State of Production Kubernetes” research, VM adoption has been a significant barrier limiting Kubernetes adoption. The study found that “continued use of VMs alongside containers” and “cultural resistance to change” remain key factors that influence how organizations approach platform adoption.

The operational differences between VM management and Kubernetes native practices create specific challenges. Traditional VM operations typically involve GUI-based tools, ticket-driven provisioning, and careful change management processes. Kubernetes operations emphasize declarative configuration, automated deployment, and rapid iteration cycles.

Consider how resource requests differ between these approaches:

- In traditional VM environments, developers typically submit requests for new virtual machines through service desk systems. Infrastructure teams review specifications, provision VMs from standard templates, and provide access after approval processes that may take days or weeks.
- In Kubernetes environments, developers modify resource specifications in YAML files, commit changes to version control, and watch automated systems deploy new configurations within minutes. This shift represents more than improved speed: It changes who makes decisions, how resources are allocated, and where operational expertise resides.

Teams accustomed to VM operations may find the Kubernetes approach initially unsettling. The increased

autonomy of developers and reduced centralized control can feel like a loss of oversight. However, this transition often leads to improved agility and more efficient resource utilization once teams adapt their processes.

“While the applications inside VMs may remain unchanged, the VMs themselves can be treated as cattle rather than pets.

Application Deployment Patterns

Moving VM workloads to Kubernetes also affects how applications are deployed and managed. Traditional VM deployments often treat virtual machines as long-lived infrastructure that hosts applications for months or years. Teams carefully configure operating systems, install dependencies, and tune performance settings specific to each workload.

KubeVirt enables a different approach where VMs become more like containers in terms of life-cycle management. While the applications inside VMs may remain unchanged, the VMs themselves can be treated as [cattle rather than pets](#). This enables practices such as automated replacement, rolling updates, and dynamic scaling, which are common in containerized environments.

This shift requires teams to reconsider how they handle VM configuration, application deployment, and operational maintenance. IaC practices become more relevant as VM specifications are defined in version-controlled manifests rather than manual configuration processes.

Building Transition Readiness

The transition also affects how teams approach monitoring, logging, and troubleshooting. Kubernetes native tooling provides new capabilities for observability and automation, but teams need to learn how these tools apply to VM workloads running under KubeVirt.

Successful VM-to-Kubernetes transitions require an honest assessment of current capabilities and systematic development of new skills. Organizations should evaluate both technical readiness and team preparedness for operational changes.

Technical readiness includes understanding how existing VM workloads will function in Kubernetes environments, identifying dependencies on VM-specific features, and planning for networking and storage requirements. Teams need to assess which applications are suitable for immediate migration versus those requiring additional preparation.

Team preparedness involves evaluating comfort levels with declarative configuration, version control workflows, and automated deployment practices. Even organizations with strong expertise using Kubernetes for containers may find that extending these practices to VMs requires additional training and process development.

“The goal is not to completely abandon proven operational practices but to thoughtfully integrate VM management into existing cloud native workflows.

Operational Practice Evolution

The goal is not to completely abandon proven operational practices but to thoughtfully integrate VM management into existing cloud native workflows. This creates unified operations that leverage the best aspects of both traditional reliability practices and modern automation capabilities.

“VMware and Kubernetes environments support multiple operational approaches, but they differ in their primary focus and default practices.

Both VMware and Kubernetes environments support multiple operational approaches, but they differ in their primary focus and default practices. VMware environments typically start with GUI-based management through vCenter, then evolve toward automation using PowerShell, PowerCLI, Ansible, or Terraform as organizations mature their practices. Many experienced VMware teams have extensive automation frameworks and IaC implementations in place.

Kubernetes takes the opposite approach. It begins with API-first, declarative operations and adds GUI tools for specific use cases, such as troubleshooting and visibility. The platform assumes automation from the start, with manual operations serving as exceptions rather than the primary workflow.

This difference affects how teams approach operational tasks and build expertise. VMware practitioners often develop deep GUI knowledge first, then gradually adopt automation tools.

Skills Assessment and Team Structure

Kubernetes practitioners typically begin with YAML manifests and command-line tools, and then add graphical interfaces to meet specific operational needs.

Organizations moving VM workloads to Kubernetes require teams that are comfortable with both approaches.

VMware expertise remains valuable for understanding application requirements, networking needs, and performance characteristics.

Kubernetes expertise provides the automation frameworks and operational practices needed for cloud native management.

Most organizations already running Kubernetes have established platform teams responsible for cluster operations, monitoring, and developer enablement. These teams typically include platform engineers, site reliability engineers (SREs), and DevOps specialists with strong automation and cloud native skills.

Separately, infrastructure operations teams manage the underlying compute, storage, and networking resources. These teams often include VMware administrators who understand hypervisor operations, storage area networks, and traditional data center technologies.

Moving VM workloads from VMware to Kubernetes affects both teams but doesn't necessarily eliminate the need for infrastructure expertise. Physical servers, storage systems, and network infrastructure require ongoing management regardless of the virtualization layer.

Skill Gap Analysis

The transition primarily affects operational workflows rather than fundamental technical knowledge. Key areas for assessment include:

Declarative Configuration

Teams need comfort with YAML manifests and version-controlled infrastructure definitions. VMware administrators with existing automation experience often adapt quickly to Kubernetes declarative models.

Container and Pod Concepts

Understanding how VMs relate to pods, containers, and Kubernetes networking requires specific training, even for experienced infrastructure professionals.

Kubernetes Tooling

Familiarity with `kubectl`, Helm, and Kubernetes native monitoring tools becomes essential for day-to-day operations.

API-First Workflows

Kubernetes emphasizes programmatic interfaces over graphical tools, requiring comfort with REST APIs and command-line operations.

Organizational Considerations

Migrations from vSphere to Kubernetes typically originate from senior leadership decisions rather than bottom-up technical preferences. CIOs or CTOs make strategic calls to reduce VMware licensing costs or standardize on cloud native platforms.

This top-down approach affects team dynamics differently than grassroots technology adoption. Infrastructure teams may need to adapt their roles rather than compete with platform teams. The goal becomes integration and skill development rather than territorial disputes.

Some VMware administrators may transition toward Kubernetes operations, while others may focus on underlying infrastructure management. Both skill sets remain valuable as organizations need expertise across the entire technology stack.

Building Migration Readiness

Establishing migration readiness ensures that both the platform and the organization are prepared for a smooth transition. Once foundational capabilities and team alignment are in place, the next step is to execute the migration using structured approaches and validated tools.

Platform Integration

Assess how KubeVirt will integrate with existing Kubernetes clusters and operational practices. This includes storage integration, networking configuration, and monitoring setup.

Application Compatibility

Evaluate which VM workloads are suitable for immediate migration versus those requiring additional preparation or application changes.

Operational Tooling

Plan how existing monitoring, backup, and management tools will work with VM workloads running under Kubernetes.

Cross-Training Programs

Provide Kubernetes training for infrastructure teams and VMware context for platform teams. Both perspectives contribute to successful VM operations.

Pilot Projects

Start with noncritical workloads to build experience and confidence before migrating business-critical applications.

Documentation and Processes

Develop new operational procedures that combine VMware expertise with Kubernetes practices.

Change Management

Explain how the migration aligns with organizational strategy and how different teams contribute to success.

Role Evolution

Help team members understand how their expertise applies in the new environment and what additional skills they need to develop.

Success Metrics

Define clear measures for migration success that acknowledge both technical achievements and team development progress.

The goal is creating unified operations that leverage the best aspects of both traditional infrastructure knowledge and modern cloud native practices.

“Technology provides capabilities, but people determine outcomes.”

Change Management by Role

Moving VM workloads to Kubernetes affects team members differently based on their current responsibilities.

VMware administrators face the most significant changes as their primary platform shifts to Kubernetes. They need comprehensive KubeVirt training, hands-on experience with declarative configuration, and gradual transition of responsibilities from VMware to Kubernetes VM management.

Platform engineers and SREs already managing Kubernetes clusters will extend their scope to include VM workloads. They need KubeVirt-specific training and cross-training with VMware administrators to understand application requirements.

Application owners may need to adapt deployment workflows but typically see minimal changes to their applications.

Network engineers need focused training on Kubernetes networking models and integrating CNI with traditional networking approaches.

Enterprise architects require strategic education on unified platform design patterns and capacity planning models.

Security teams need training on Kubernetes-specific security tools while applying existing security principles.

Roles like **vendor management**, **procurement teams**, and **infrastructure hardware specialists** see minimal operational impact. Hardware teams continue managing servers and storage regardless of the virtualization layer. Vendor relationships may shift as organizations reduce VMware licensing and adopt Kubernetes-related services.

Create role-specific training programs rather than generic Kubernetes education. Establish cross-functional working groups to solve migration challenges while building understanding across team boundaries. Provide realistic lab environments where teams can practice with their specific tools and scenarios.

Address role-specific concerns directly. VMware administrators may worry about skill obsolescence while developers may fear increased complexity. Show how existing expertise remains valuable while demonstrating clear development paths

through pilot projects and success stories from similar organizations. Focus on evolution rather than replacement, building on existing strengths while developing capabilities that serve strategic objectives.



SUMMARY

Spectro Cloud delivers simplicity and control to organizations running Kubernetes at any scale.

With its Palette platform, Spectro Cloud empowers businesses to deploy, manage, and scale Kubernetes clusters effortlessly — from edge to data center to cloud — while maintaining the freedom to build their perfect stack.

Trusted by leading organizations worldwide, Spectro Cloud transforms Kubernetes (K8s) complexity into elegant, scalable solutions, enabling customers to master their cloud native journey with confidence.

Spectro Cloud is a Gartner Cool Vendor, CRN Tech Innovator, and a “leader” and “outperformer” in GigaOm’s 2025 Radars for Edge Kubernetes and Managed Kubernetes.

Learn more and get started at spectrocloud.com

KEY FEATURES

Run your VMs on Kubernetes: Activate a complete enterprise-grade KubeVirt stack on your chosen clusters, and run your VMs as first-class citizens alongside containers, in the bare metal data center, or at the edge, with all the same life-cycle controls.

The benchmark for choice and flexibility: Choose the K8s stack that’s right for you, from operating system and distro to add-ons and apps. Unify K8s management across clouds, edge, airgapped, virtualized, and bare-metal data centers. We won’t constrain you.

Complete life cycle management: Declarative management for the full K8s stack. Automate away Day 2 toil like fleet upgrades, policy enforcement, and backups for multicluster environments scaling to tens of thousands of clusters.

Mapping the VMware Stack to Cloud Native Equivalents

Mapping VMware Components to Cloud Native Equivalents				
VMware Component	Function	Cloud Native Equivalent	Key Differences	Cloud Native Limitations
vCenter Server	Centralized management interface	Kubernetes API Server + kubectl	API-first approach vs. GUI-first approach	Kubernetes has no single official GUI; management is often via CLI + multiple dashboards.
ESXi Hypervisor	Hardware virtualization	Kubernetes worker nodes with OS running KVM	Workers run containers by default; KVM adds VM support	Without KubeVirt, Kubernetes does not provide virtualization.
vSphere VMs	Virtual machines	KubeVirt VMs/ pods	KubeVirt for VMs, pods for containers	KubeVirt VMs require KVM-enabled nodes.
Templates	VM templates	KubeVirt VM template CRDs + CDI data volumes	YAML-defined templates and disk import/cloning	CDI focuses on disk/image handling, not full-featured VM templating like VMware.
Snapshots	Point-in-time VM state	CSI volume snapshots + VM pause state	Storage-level vs. full VM+memory snapshots	CSI snapshots alone do not capture CPU/ memory state unless combined with VM pause.

Mapping VMware Components to Cloud Native Equivalents (cont.)				
VMware Component	Function	Cloud Native Equivalent	Key Differences	Cloud Native Limitations
Resource Pools	Resource allocation	Namespaces + ResourceQuotas + LimitRanges + RBAC	Kubernetes native multitenancy	Namespaces provide logical isolation; resource guarantees differ from VMware's resource pools.
Folders and Tags	Organization and metadata	Labels, annotations, namespaces	Flat key-value metadata vs. hierarchical folder/tag model	No built-in hierarchical view; relies on external tooling.
vMotion	Live VM migration	KubeVirt live migration/pod rescheduling	Declarative migration vs. imperative migration	Live migration requires shared storage or block migration; not as seamless as vMotion.
Distributed Resource Scheduler (DRS)	Automatic resource balancing	Kubernetes scheduler + descheduler	Policy-based scheduling vs. rule-based scheduling	DRS is more granular for VM workloads; Kubernetes requires combining components.
vSAN	Distributed storage	Rook-Ceph/ Portworx/ Longhorn	CSI-based integration vs. proprietary integration	Operational complexity; feature set differs per project.
Storage Policies	Storage service levels	StorageClass	YAML-defined vs. GUI-configured	Operational complexity; feature set differs per project.
NSX	Software-defined networking	Calico/Cilium/ Antrea + service mesh	Multiple components vs. single product	Limited dynamic enforcement; depends on underlying CSI driver capabilities.

Mapping VMware Components to Cloud Native Equivalents (cont.)				
VMware Component	Function	Cloud Native Equivalent	Key Differences	Cloud Native Limitations
NSX Distributed Firewall	Micro-segmentation	Network policies + service mesh policies	Label-based rules vs. IP-based rules	More fragmented architecture; needs integration work.
NSX Load Balancer	L4-L7 load balancing	NGINX/Traefik/HAProxy	Ingress resources vs. virtual servers	Service mesh adds L7 policies, but complexity is higher.
Multiple vNICs	Multiple network interfaces (NICs)	Multus CNI	Meta-plugin enabling multiple CNIs	Multus requires extra CNI plugins; not native in Kubernetes.
Aria Operations	Monitoring and analytics	Prometheus + Grafana + AlertManager	Composable stack vs. integrated suite	Requires integration; lacks single-vendor support like VMware vROps.
Aria Log Insight	Log aggregation	EFK/ELK Stack/ Loki + Grafana	Open source vs. proprietary	Needs more tuning for performance and retention.
Aria Automation	Infrastructure automation	Crossplane + Argo CD/Flux (GitOps)	Git-based vs. workflow-based	Crossplane covers provisioning, Argo CD covers app delivery; combined effort needed.
Site Recovery Manager (SRM)	Disaster recovery (DR)	CloudCasa/ Kasten K10/ Portworx DR	Backup-focused vs. replication-focused	Kubernetes DR is less mature; SRM's orchestration features are more robust.

Although the mapping above may seem complex and overwhelming, tools are being developed to map the components and ease the transition.

VMware Migration Operator (VMO) is one such tool specifically designed to

help organizations migrate from vSphere to cloud native environments. One of VMO’s unique features is a GUI that maps major vCenter functions, making it easier for vSphere administrators to manage KubeVirt environments and VMs.



FIG 4.1 Palette GUI with a VMO deployed and active. The lower-right section shows all the cluster profiles that must be deployed for this example cluster to fully function. You can also see running services, cluster health, and other key facts for this bare metal cluster with Canonical MaaS.



FIG 4.2 Drilling down into a VM’s details enables you to see the VM console, take snapshots, or perform diagnostics.

The transition from managing VMs to running them in Kubernetes ultimately succeeds or fails based on how well organizations manage the human dimensions of change. Technology provides capabilities, but people determine outcomes. Organizations that invest in understanding cultural differences, building capable teams, and managing change thoughtfully position themselves not just to

complete a migration but to thrive in a cloud native future.

The journey requires patience, persistence, and recognition that transforming technology is far easier than changing culture. Still, the rewards of successful transformation extend far beyond escaped licensing costs to encompass true organizational agility and innovation capability.

CHAPTER 5

Tools and Techniques for Running VMs on Kubernetes

Organizations already using Kubernetes face a practical challenge when dealing with VM-based workloads. Whether these are legacy applications, commercial software packages, or systems that simply cannot be containerized, teams need reliable ways to bring these VMs into their Kubernetes environments. This chapter examines the tools and techniques available for running VMs on Kubernetes, helping teams select the right approach for their specific workloads.

Assessing Your VM Workloads

Before selecting tools, evaluate each VM-based application individually. Consider these key questions for each workload:

Can this VM run on KubeVirt?

Most standard Linux and Windows VMs can run on KubeVirt with minimal modification. Check for any specialized hardware requirements or kernel dependencies that might prevent migration.

Should it stay on VMware?

Some workloads may have vendor support requirements or technical constraints that mandate VMware. Document these exceptions but don't let them block progress on other workloads.

Is there a containerized alternative?

Before migrating a VM, check if a cloud native replacement already exists. Many commercial applications now offer container-based versions that integrate better with Kubernetes.

Can it be retired?

Migration projects offer natural opportunities to decommission outdated systems. Review usage patterns and business requirements to identify candidates for retirement.

Does it need refactoring?

While this book focuses on running VMs on Kubernetes, some applications may benefit from containerization if development resources are available.

Migration Approaches for VM Workloads

Migrating VM workloads to Kubernetes involves selecting a strategy that aligns with operational requirements, application architecture, and modernization objectives. The following approaches outline different paths based on the level of modification and integration with Kubernetes native capabilities.

Rehosting: The Fastest Path

Rehosting moves VMs to Kubernetes with minimal changes, preserving operating systems, configurations, and application architectures. This approach works best for:

- Commercial software that cannot be modified.
- Legacy applications where expertise or source code is unavailable.
- Windows workloads with deep OS dependencies.

- Applications that must maintain specific compliance certifications.

The primary tool for rehosting is KubeVirt, which extends Kubernetes to run traditional VMs. Through KubeVirt, VMs become standard Kubernetes resources, managed with the same tools and workflows as containerized applications.

Replatforming: Gradual Modernization

Some teams choose to make moderate modifications during migration to better leverage Kubernetes capabilities. This might include:

- Replacing file-based configuration with ConfigMaps.
- Adopting Kubernetes services for load balancing.

- Implementing horizontal pod autoscaling for stateless components.
- Breaking monolithic VMs into a few larger services.

This approach works well when teams have some flexibility to modify applications but lack resources for complete refactoring.

Kubernetes Native Tools: KubeVirt and CDI

KubeVirt

KubeVirt seamlessly extends Kubernetes with CRDs that describe VM concepts like disks, network interfaces, and machine types. Teams can define VMs using standard Kubernetes YAML manifests, applying familiar declarative patterns.

Converting VMs starts with disk images. The `qemu-img` tool transforms

VMware VMDK files into QCOW2 or raw formats that KubeVirt consumes. This process preserves the complete file system and installed software while adapting the format for Kubernetes.

Containerized Data Importer (CDI)

CDI automates importing VM disk images into Kubernetes storage. It supports multiple sources including HTTP endpoints, AWS S3-compatible storage, and direct uploads. CDI handles format conversion automatically and provides smart cloning capabilities, using copy-on-write (COW) techniques to rapidly provision multiple VMs from golden images.

The `DataVolume` abstraction combines `PersistentVolumeClaims` with import specifications, allowing teams to declare both storage requirements and source data in a single resource.

Migration Automation Tools: Forklift and VM Migration Assistant

Network and Storage Considerations

Network configuration requires mapping VMware port groups to Kubernetes network attachment definitions through Multus CNI. Teams must translate firewall rules to NetworkPolicies and ensure IP addressing aligns with enterprise requirements.

Storage migration involves provisioning PersistentVolumes with appropriate storage classes and validating performance characteristics. While Kubernetes storage providers offer capabilities similar to VMware through CSI, careful planning ensures applications maintain required performance levels.

Forklift: Open Source Migration

Forklift automates the migration process from VMware to KubeVirt. Developed by the Konveyor community with contributions from Red Hat, it provides:

Automated discovery and

assessment: Inventories VMware infrastructure and analyzes workload characteristics to identify compatibility issues before migration begins.

Warm migration capabilities: Copies VM disks while source VMs continue running, using changed block tracking to minimize downtime during cutover.

Flexible network mapping: Handles complex network transformations while preserving IP addresses when required or integrating with Kubernetes IP management.

Storage optimization: Converts thick-provisioned VMware disks to thin-provisioned Kubernetes volumes, potentially reducing storage costs.

As an open source tool, Forklift offers transparency and freedom from vendor lock-in. Organizations can inspect code, contribute improvements, and customize for specific requirements. Support comes primarily through community channels, requiring teams to develop internal expertise.

Spectro Cloud VM Migration Assistant

Spectro Cloud provides a commercial migration solution based on Forklift, which is integrated with its Palette platform. Key differentiators include:

Enterprise governance: Migrated VMs automatically inherit policies for backup, monitoring, and access control through Palette’s cluster profiles.

Business continuity features:

Supports migration waves for related applications, detailed scheduling aligned with maintenance windows, and rollback procedures.

Multi-environment flexibility: Migrates workloads to on-premises clusters,

public cloud Kubernetes services, or edge locations through a single platform.

Professional support: Commercial backing with professional services for migration planning and support agreements for rapid issue resolution.

The tool handles complex scenarios like multitier applications with interdependencies and migrations spanning multiple data centers. Performance optimization features analyze utilization patterns to recommend rightsizing opportunities.

Selecting the Right Tool

Tool selection depends on several factors:

Scale and complexity: Native tools work well for small numbers of VMs or learning exercises. Larger migrations benefit from automation through Forklift or commercial solutions.

Support requirements: Organizations with limited Kubernetes expertise or business-critical workloads may prefer commercial tools with professional support.

Customization needs: Open source tools like Forklift offer maximum flexibility for organizations with specific requirements.

Budget constraints: Native tools and Forklift avoid licensing costs, while some commercial solutions provide additional features and support at a price.

Integration requirements: Consider how migration tools integrate with existing monitoring, backup, and security systems.

A Practical Migration Process

Regardless of tool choice, successful VM migration follows a consistent pattern:

1. **Inventory and assess:**
Document all VM workloads and their requirements.
2. **Categorize workloads:**
Determine which VMs to rehost, replace, or retire.
3. **Plan migration waves:** Group related applications for coordinated migration.
4. **Prepare the target environment:** Configure storage classes, networks, and policies.
5. **Execute pilot migrations:** Test processes with noncritical workloads.
6. **Migrate production workloads:** Follow established patterns from pilot phase.
7. **Validate functionality:** Verify application behavior and performance.
8. **Optimize post-migration:** Rightsize resources based on actual utilization.

Key Takeaways

Running VMs on Kubernetes is a practical solution for organizations that need to maintain VM-based workloads while standardizing on Kubernetes. KubeVirt provides the foundation, while tools like CDI, Forklift, and commercial solutions add automation and enterprise features. Success depends on careful workload assessment, appropriate tool selection, and systematic execution rather than attempting wholesale platform transformation.

Focus on moving workloads that benefit most from Kubernetes integration while accepting that some systems may need to remain on traditional virtualization platforms. The goal is not universal migration but rather bringing the right workloads into your Kubernetes environment to maximize operational efficiency and minimize platform sprawl.

“Success depends on careful workload assessment, appropriate tool selection, and systematic execution rather than attempting wholesale platform transformation.”

CHAPTER 6

Day 2 Operations and Optimization

Running VMs on Kubernetes requires ongoing operational practices that differ from managing either traditional VMs or pure containerized workloads. This chapter examines the operational realities of maintaining VM workloads alongside containers in production Kubernetes environments.

Unified Management for Mixed Workloads

Managing VMs through Kubernetes delivers practical benefits for teams already operating containerized workloads. The same `kubectl` commands, YAML manifests, and GitOps workflows that manage containers now also manage VMs. This consistency reduces context switching and accelerates troubleshooting.

A single observability stack covers both workload types. Prometheus collects metrics from VMs and containers alike. Grafana dashboards display unified views of resource consumption. Distributed tracing tools like Jaeger track requests across mixed architectures. When performance issues arise, teams investigate using familiar tools regardless whether components run as VMs or containers.

Resource scheduling improves when Kubernetes manages the entire infrastructure pool. When the scheduler makes placement decisions, it considers all workloads together, enabling better resource utilization. Organizations often see improvements in infrastructure density compared to

running separate clusters for VMs and containers.

Security policies apply consistently across workload types. NetworkPolicies control traffic between VMs and containers. RBAC mechanisms grant permissions uniformly. Audit logs capture all changes through a single API. This consistency simplifies compliance reporting while reducing the risk of configuration gaps between systems.

Applying Kubernetes Practices to VM Workloads

VMs running on Kubernetes benefit from established Kubernetes operational patterns. GitOps workflows that teams already use for containers extend naturally to VM management. Configuration changes follow the same path through version control and automated deployment. Tools like Argo CD and Flux monitor repositories and apply VM configuration updates automatically.

Progressive delivery becomes possible for VM-based applications. Teams can implement canary deployments for VMs, routing small percentages of traffic to updated versions. Blue-green deployments enable rapid rollback if issues emerge. While VMs lack the instant startup times of containers, these patterns still reduce deployment risk compared to traditional VM update approaches.

Declarative configuration brings self-healing capabilities to VM workloads. When Kubernetes detects configuration drift, it automatically reconciles the actual state with the desired state. If a node hosting VMs fails, Kubernetes

reschedules them elsewhere. These behaviors reduce manual intervention requirements.

The challenge lies in adapting these practices to VM characteristics. VM startup times affect autoscaling effectiveness. Large VM images affect deployment speed. Teams must tune readiness probes and scaling policies to account for these differences while still leveraging Kubernetes automation.

Monitoring and Optimization Strategies

Performance monitoring for VMs in Kubernetes requires attention to both VM-specific and Kubernetes native metrics. Traditional VM metrics like CPU steal time and memory ballooning remain relevant. These combine with Kubernetes metrics like pod resource requests and limits to provide complete visibility.

Cost optimization opportunities emerge from unified resource management. Teams can identify overprovisioned VMs by comparing actual usage against resource requests. Namespace-based quotas enable chargeback models. Spot instances or preemptible nodes can host noncritical VM workloads, reducing costs without sacrificing reliability for production systems.

Capacity planning benefits from Kubernetes native approaches. Historical metrics feed predictive models that forecast resource needs. Cluster autoscaling provisions nodes before VMs require them. This proactive approach prevents performance degradation during demand spikes.

Application startup optimization becomes critical for VM workloads that need to scale. Teams optimize VM images by removing unnecessary packages and services. Init containers can prewarm caches or establish database connections. JVM tuning reduces initialization time for Java applications. These optimizations make VM-based applications more responsive to scaling events.

Working With VMs Within the Kubernetes Ecosystem

The Kubernetes ecosystem provides tools that enhance VM operations without requiring custom development. These tools work equally well for VMs and containers, providing consistent operational experiences.

Backup and recovery: Disaster recovery is simpler compared to traditional VM backup and recovery solutions. Velero backs up VM definitions and persistent volumes together, helping to ensure application-consistent snapshots. Then, recovery procedures can restore entire applications including VM configurations and data.

Security scanning: Container security tools scan VM images for vulnerabilities, while admission controllers enforce security policies before VMs deploy, and runtime security tools detect anomalous behavior in running VMs. These capabilities also fold neatly into existing security workflows.

Developer platforms: Internal developer platforms abstract Kubernetes complexity for teams

deploying VM-based applications.

These platforms encode organizational standards and provide simplified interfaces, enabling developers without deep Kubernetes knowledge to deploy VMs while operations teams maintain governance.

Service mesh integration: Service mesh tools such as Istio help teams manage traffic for VM workloads. These tools provide sophisticated routing, security, and observability features, and enable VMs to participate fully in zero trust networking alongside their containerized counterparts.

Operational Considerations for VM Workloads

Running VMs on Kubernetes introduces specific operational considerations that teams must address. It's critical to understand these considerations to set appropriate expectations and plan for success.

Storage performance often differs between Kubernetes and traditional virtualization platforms. Benchmark I/O-intensive VM workloads early to identify potential bottlenecks, as some workloads may require specific storage classes or node configurations to meet performance requirements.

Network configuration complexity increases when VMs require specific IP addresses or VLAN access. Multus and similar CNI plugins provide these capabilities but require careful planning. For this reason, document network requirements and test configurations thoroughly.

Resource overhead for running VMs through KubeVirt affects capacity planning. The additional abstraction layer consumes resources that pure containers avoid; you must account for

this overhead when sizing clusters and setting resource requests.

Migration between nodes takes longer for VMs than containers due to image sizes and state management. This affects maintenance windows and failure recovery times. Plan accordingly and set appropriate expectations with stakeholders.

Key Takeaways

Operating VMs on Kubernetes combines the benefits of unified management with the flexibility to run workloads that cannot be containerized. Success requires adapting Kubernetes practices to VM characteristics while leveraging ecosystem tools that work across workload types.

Focus operational efforts on areas where VMs on Kubernetes provide clear value, such as unified observability, consistent security policies, and integrated deployment pipelines. Accept that some operational aspects, like startup times and migration speed, will differ from pure container workloads.

The goal is not to make VMs behave exactly like containers but to operate them effectively within the Kubernetes ecosystem. Teams that embrace both the opportunities and constraints of running VMs on Kubernetes build more flexible and maintainable platforms.

“Teams that embrace both the opportunities and constraints of running VMs on Kubernetes build more flexible and maintainable platforms.”

CHAPTER 7

Conclusion and Key Takeaways

Running VMs on Kubernetes solves practical problems that most organizations face. Not every workload can be containerized. Commercial software often requires specific operating systems. Legacy applications lack the source code or expertise for refactoring. These realities make KubeVirt essential for modern Kubernetes platforms.

The Reality of Mixed Workloads

For Kubernetes teams, VMs represent the remaining workloads that cannot easily transition to — and haven't already been moved into — containers. These typically include commercial off-the-shelf software with rigid OS requirements, Windows Server applications with deep system dependencies, and legacy applications that would require a full rewrite to be properly containerized.

KubeVirt makes these workloads manageable within your existing Kubernetes infrastructure. VMs become standard Kubernetes resources, managed through the same APIs and tools as your containerized applications. This eliminates the need to maintain separate virtualization platforms for a handful of stubborn applications.

Practical Implementation

1. Focus on the workloads that benefit from Kubernetes integration such as applications that need to communicate with containerized services or workloads that your team wants to manage through existing Kubernetes workflows.
2. Plan for the operational differences. Workloads running in VMs have different attributes, characteristics, and requirements for resources. Set appropriate resource requests and limits. Configure readiness probes with longer timeouts. Adjust autoscaling policies to account for slower startup times.
3. Consider migration timing and approach. Running VMs on Kubernetes works best when teams already have mature Kubernetes operations. If your platform team struggles with basic container management, adding VMs will increase complexity. Build expertise with containerized workloads first.

Key Technical Considerations

1. **Resource planning:** VMs on Kubernetes consume more resources than equivalent containers. Budget for the overhead of running KubeVirt components and the virtualization layer, and plan storage carefully since VM images are larger than container images.
2. **Networking complexity:** VMs often require specific IP addresses or VLAN access. Implement Multus or similar CNI plugins before migrating network-sensitive workloads. Make sure to test network configurations thoroughly in development environments.
3. **Performance expectations:** Accept that VMs will not perform identically to bare metal or traditional hypervisors. Benchmark critical workloads early to identify any performance gaps, and recognize that some I/O-intensive applications may not be suitable candidates.

4. **Tool selection:** Choose migration tools based on your scale and expertise. Native tools work for small numbers of VMs, while Forklift provides automation for larger migrations. Commercial solutions offer support for complex enterprise scenarios.

“Teams that approach VMs on Kubernetes with clear objectives and realistic expectations build more maintainable and efficient platforms.”

Operational Best Practices

1. Apply Kubernetes patterns where they make sense for VMs. Use GitOps workflows to manage VM configurations effectively. Integrate VM metrics into your monitoring stacks that already track containers, and apply security policies uniformly across workload types.
2. Recognize where VMs require different treatment. Backup strategies must account for larger data volumes. Therefore, build longer recovery time objectives into your disaster recovery procedures, and consider VM migration times in your maintenance windows.
3. Build team expertise gradually. Start with noncritical VMs to develop operational experience. Document procedures specific to VM management, and train team members on troubleshooting VM-specific issues within Kubernetes.

Frequently Asked Questions

Can internal teams handle VM migration?

Teams familiar with Kubernetes can typically handle straightforward VM migrations using tools like Forklift. However, complex scenarios involving hundreds of VMs or critical production workloads may benefit from professional services.

How long do migrations take?

Individual VM migrations take hours to days depending on size and complexity. Portfolio-wide migrations typically span months, with most time spent on planning and testing rather than actual migration.

How automated is the migration process?

Migration tools automate disk conversion and resource creation. Teams still need to validate network configurations, test application functionality, and optimize resource allocations manually.

How should application teams be involved?

Application teams must participate in testing and validation. They need to understand any behavioral changes and adjust their operational procedures for the Kubernetes environment.

How do we manage risk?

Start with development environments. Maintain rollback plans for each migration, and keep source VMs available until production validation completes. Consider parallel running for critical applications.

Moving Forward

Running VMs on Kubernetes is not about transforming your entire infrastructure or escaping vendor lock-in. It solves the specific problem of managing non-containerizable workloads within your Kubernetes platform. Success comes from carefully selecting which VMs to migrate, setting realistic expectations about operational characteristics, and gradually building team expertise.

Focus on workloads where Kubernetes management provides clear value. Accept that some applications may remain on traditional virtualization platforms. The goal is operational simplicity through platform consolidation where it makes sense, not universal migration for its own sake.

Teams that approach VMs on Kubernetes with clear objectives and realistic expectations build more maintainable and efficient platforms. They avoid the complexity of managing multiple infrastructure types while accepting the operational realities of running VMs within Kubernetes.

For Further Reading

1. [KubeVirt User Guide](#)
A comprehensive guide to learning and using KubeVirt.
2. [Palette VMO Reference Architecture](#)
Reference architecture for Palette Virtual Machine Orchestrator (VMO) for bare metal environments.
3. [Kubernetes at the Edge: Container Orchestration at Scale](#)
Strategies to navigate the complexities, evaluate the vendors, and master the best practices to deploy and manage Kubernetes at the edge.

About the Author



Janakiram MSV (Jani) is a practicing architect, research analyst, and advisor to Silicon Valley startups. He focuses on the convergence of machine intelligence and modern infrastructure powered by the cloud native technology stack. Before becoming an entrepreneur, he spent over a decade as a product manager and technology evangelist at Microsoft and Amazon Web Services. Janakiram regularly writes for Forbes, InfoWorld, and The New Stack, covering the latest from the technology industry. He is a keynote speaker for internal sales conferences, product launches, and user conferences hosted by technology companies of all sizes.

Disclosure

The following companies mentioned in this ebook are also sponsors of The New Stack: Amazon Web Services (AWS), Cloud Native Computing Foundation (CNCF), Nutanix, Red Hat, and VMware by Broadcom.

